

SketchSense — Wissenschaftlicher Projektbericht

Projekt: Echtzeit-Skizzenerkennung im Browser mittels Convolutional Neural Network

Datensatz: Google Quick, Draw! Dataset

Plattform: MacBook, 8 GB RAM, Apple M-Chip (CPU-Training, kein dediziertes GPU)

Trainingszeitraum: 16.–17. Mai 2026

1. Motivation & Zielsetzung

Ziel des Projekts ist die Entwicklung eines Systems, das handgezeichnete Skizzen in Echtzeit klassifizieren kann — vollständig im Browser, ohne Backend-Server. Der Nutzer zeichnet auf einem Canvas, das Modell gibt kontinuierlich Vorhersagen aus.

Skizzenerkennung ist eine anspruchsvolle Aufgabe. Im Gegensatz zu Fotos sind Skizzen abstrakt, stark variierend und enthalten kaum Texturinformationen. Eitz, Hays & Alexa (2012) zeigen, dass Menschen bei der Klassifikation von Skizzen selbst nur ~73 % Genauigkeit erreichen — ein wichtiger Benchmark. Neuere CNN-basierte Ansätze wie Sketch-a-Net (Yu et al., 2015) übertreffen menschliche Performance auf dedizierten Sketch-Datensätzen.

Das Zielspektrum für dieses Projekt:

Accuracy	Bewertung
≥ 80 %	Ausreichend
≥ 85 %	Gut
≥ 90 %	Sehr gut

2. Datensatz

2.1 Google Quick, Draw! Dataset

Der verwendete Datensatz stammt aus dem Google-Projekt *Quick, Draw!* (Jongejan et al., 2016), einem Browser-Spiel bei dem Millionen von Nutzern Skizzen zu vorgegebenen Begriffen zeichnen. Das daraus resultierende Dataset enthält über 50 Millionen Zeichnungen in 345 Kategorien und ist frei verfügbar [1].

Ha & Eck (2017) nutzen denselben Datensatz zur Entwicklung von Sketch-RNNs — einem generativen Modell für Skizzen [2]. Für diese Arbeit wird die vorverarbeitete Variante als 28×28-Pixel-Bitmaps (`.npy`-Format) verwendet.

2.2 Klassenauswahl

Es wurden **100 Kategorien** aus insgesamt 345 verfügbaren ausgewählt. Ziel war eine hohe visuelle Unterscheidbarkeit zwischen den Klassen. Ähnliche oder leicht verwechselbare Kategorien (z. B. *bird*, *parrot*, *duck*, *flamingo*, *swan* gemeinsam) wurden auf die eindeutigsten reduziert.

Die 100 gewählten Klassen verteilen sich auf 9 thematische Gruppen:

Gruppe	Anzahl	Beispiele
Tiere	22	ant, bear, butterfly, dolphin, elephant, shark, whale, zebra
Fahrzeuge	10	airplane, bicycle, car, helicopter, train, truck
Natur	12	cactus, cloud, lightning, mountain, rainbow, sun, tree
Essen	12	apple, cake, donut, hamburger, pizza, strawberry
Objekte	15	book, clock, hammer, scissors, shoe, umbrella
Gebäude	8	barn, castle, church, house, lighthouse, skyscraper
Instrumente	8	cello, drums, guitar, piano, violin
Sport	6	basketball, hockey stick, skateboard, soccer ball, yoga
Sonstiges	7	crown, diamond, hourglass, snowman, star

2.3 Datenumfang

Parameter	Wert
Klassen	100
Samples pro Klasse	25.000
Gesamtdatensatz	2.500.000 Zeichnungen
Bildgröße	28 × 28 Pixel, Graustufen
Trainingsset	1.875.000 (75 %)
Validierungsset	375.000 (15 %)
Testset	250.000 (10 %)

3. Datenvorbereitung & Vorverarbeitung

3.1 RAM-Optimierung

Ein zentrales Problem bei 8 GB RAM ist die Größe des Datensatzes. Naive Speicherung aller Bilder als `float32` würde ~7,8 GB benötigen — zu viel für parallelen Betrieb mit Betriebssystem und TensorFlow.

Lösung: Die Rohdaten werden als `uint8` (1 Byte/Pixel statt 4 Byte) gehalten:

```
uint8: 100 × 25.000 × 784 Bytes ≈ 2,0 GB  
float32: 100 × 25.000 × 784 × 4 ≈ 7,8 GB
```

Die Normalisierung (`/255` , Konvertierung zu `float32`) findet ausschließlich innerhalb der `tf.data` -Pipeline statt — jeweils nur ein Batch (128 Bilder) liegt gleichzeitig als `float32` im Speicher.

3.2 Augmentierung

Um Überanpassung zu reduzieren und die Robustheit gegenüber unterschiedlichen Zeichenstilen zu erhöhen, werden Trainingsbilder on-the-fly augmentiert:

- **RandomRotation** ±8 %
- **RandomZoom** ±8 %
- **RandomTranslation** ±5 % (horizontal & vertikal)

Augmentierung wird nur auf Trainingsdaten angewandt, nicht auf Validierungs- oder Testdaten.

3.3 Pipeline

```
Dataset → Shuffle(buffer=50.000) → Batch(128) → Normalisierung → Augmentierung → Prefetch(AU
```

4. Modellarchitektur

4.1 Architekturübersicht

Das Modell ist ein Convolutional Neural Network (CNN) mit 4 Convolutional Blöcken, inspiriert von VGGNet-Prinzipien (Simonyan & Zisserman, 2014) [3], aber stark komprimiert für Browser-Deployment.

Input: $28 \times 28 \times 1$

Block 1: Conv2D(32, 3×3) → BatchNorm → MaxPool(2×2) → Dropout(0.2)
 $28 \times 28 \rightarrow 14 \times 14$

Block 2: Conv2D(64, 3×3) → BatchNorm → MaxPool(2×2) → Dropout(0.2)
 $14 \times 14 \rightarrow 7 \times 7$

Block 3: Conv2D(128, 3×3) → BatchNorm → Dropout(0.2)
 7×7 (kein Pooling)

Block 4: Conv2D(256, 3×3) → BatchNorm → GlobalAveragePooling2D
 $7 \times 7 \rightarrow 256$

Head: Dense(512, ReLU) → Dropout(0.4) → Dense(100, Softmax)

4.2 Designentscheidungen

- **GlobalAveragePooling** statt Flatten: Reduziert Parameteranzahl und wirkt regularisierend
- **BatchNormalization** nach jedem Conv-Layer: Stabilisiert das Training, erlaubt höhere Lernrate
- **Kein Pooling in Block 3 & 4:** Bei 7×7 -Featuremaps wäre weiteres Pooling zu aggressiv
- **~573.000 Parameter:** Groß genug für 100 Klassen, klein genug für schnelle Inferenz im Browser

4.3 Parameterverteilung

Schicht	Parameter
Conv Block 1 (32 Filter)	448
Conv Block 2 (64 Filter)	18.752
Conv Block 3 (128 Filter)	74.368
Conv Block 4 (256 Filter)	296.192
Dense(512)	131.584
Dense(100)	51.300
Gesamt	572.644

5. Training

5.1 Hyperparameter

Parameter	Wert
Optimizer	Adam
Initiale Lernrate	0,001
Batch-Größe	128
Max. Epochen	40
Verlustfunktion	Sparse Categorical Crossentropy

5.2 Callbacks

EarlyStopping (`monitor=val_accuracy` , `patience=6`): Stoppt das Training wenn sich die Validierungsgenauigkeit über 6 Epochen nicht verbessert. Das beste Modell wird automatisch wiederhergestellt.

ReduceLROnPlateau (`monitor=val_loss` , `factor=0.5` , `patience=3` , `min_lr=1e-6`): Halbiert die Lernrate wenn der Validierungsverlust über 3 Epochen stagniert. Ermöglicht feinere Anpassung in späteren Trainingsphasen.

ModelCheckpoint (`monitor=val_accuracy` , `save_best_only=True`): Speichert das Modell ausschließlich wenn eine neue beste Validierungsgenauigkeit erreicht wird.

5.3 Hardware & Dauer

Training auf einem MacBook mit Apple M-Chip, 8 GB Unified Memory, CPU-basiert (kein dediziertes GPU-Training).

Metrik	Wert
Trainierte Epochen	15 (manuell gestoppt)
Ø Compute-Zeit pro Epoche	~33 Minuten
Gesamtdauer (Wanduhrzeit)	~18 Stunden

6. Ergebnisse

6.1 Trainingsläufe im Vergleich

Im Rahmen des Projekts wurden zwei Trainingsläufe durchgeführt:

	Lauf 1	Lauf 2 (final)
Klassen	150	100
Samples/Klasse	10.000	25.000
Gesamtsamples	1.500.000	2.500.000
Beste Val-Accuracy	76,5 %	82,3 %
Epoche bei Bestwert	18	13

Erkenntnis: Weniger, aber visuell distinktere Klassen mit mehr Samples pro Klasse übertrifft mehr Klassen mit weniger Daten deutlich (+5,8 Prozentpunkte). Das Modell profitiert stärker von Datentiefe als Datenbreite — konsistent mit der Beobachtung, dass Sketch-CNNs viele Varianten eines Motivs benötigen (Yu et al., 2015).

6.2 Epochenverlauf (Lauf 2)

Epoche	Train-Acc	Val-Acc	Val-Loss	Modell gespeichert
1	66,5 %	77,8 %	0,818	✓
2	73,6 %	79,8 %	0,744	✓
3	75,1 %	80,6 %	0,710	✓
4	75,9 %	80,6 %	0,709	✓
5	76,4 %	81,3 %	0,679	✓
6	76,8 %	81,8 %	0,668	✓
7	77,1 %	81,6 %	0,674	—
8	77,3 %	81,8 %	0,662	✓
9	77,5 %	81,9 %	0,662	✓
10	77,7 %	82,2 %	0,652	✓
11	77,8 %	81,9 %	0,664	—
12	77,9 %	80,9 %	0,700	—
13	78,0 %	82,3 %	0,650	✓ (Bestwert)
14	78,1 %	81,7 %	0,669	—
15	78,2 %	81,9 %	0,663	—

Training wurde nach Epoche 15 manuell gestoppt. Das gespeicherte Modell stammt aus Epoche 13.

6.3 Zusammenfassung

Metrik	Wert
Beste Validierungsgenauigkeit	82,3 % (Epoche 13)
Beste Validierungsloss	0,650
Trainingsgenauigkeit (Ep. 13)	78,0 %
Modellgröße (gespeichert)	6,9 MB

6.4 Einordnung

Die erreichte Validierungsgenauigkeit von **82,3 %** übertrifft das Mindestziel (80 %) deutlich.

Zum Vergleich: Eitz et al. (2012) berichten für Menschen bei einem ähnlichen Sketch-Klassifikationsproblem eine Erkennungsrate von ~73 %. Das hier trainierte Modell übertrifft damit die menschliche Baseline-Performance um ca. 9 Prozentpunkte. Yu et al. (2015) erzielen mit Sketch-a-Net auf dem TU-Berlin-Datensatz ~77 % bei 250 Klassen — bei 100 Klassen und dem umfangreicheren QuickDraw-Datensatz ist das hier erzielte Ergebnis von 82,3 % plausibel und wettbewerbsfähig.

7. Technische Implementierung

7.1 Tech-Stack

Komponente	Technologie
Modelltraining	Python 3.12, TensorFlow 2.19, Keras
Datenverarbeitung	NumPy, tf.data
Modellexport	TensorFlow.js (tfjs_converter)
Frontend	React 18, Vite
Inferenz	TensorFlow.js (Browser)
Deployment	Statisch, kein Backend erforderlich

7.2 Inferenz im Browser

Nach dem Training wird das `.keras` -Modell mit `tfjs_converter` in das TF.js-Format konvertiert und direkt im Browser geladen — ohne Serververbindung zur Inferenzzeit.

Preprocessing-Pipeline im Browser: 1. Canvas-Inhalt → 28×28 Pixel skalieren (Grayscale) 2.

Pixel-Inversion (QuickDraw: schwarze Striche auf weiß → invertieren) 3. Normalisierung: `/ 255`

→ Wertebereich `[0, 1]` 4. Shape anpassen: `[1, 28, 28, 1]` (Batch-Dimension) 5.

`model.predict()` → Softmax-Wahrscheinlichkeiten → Top-5 anzeigen

Die Inferenzlatenz im Browser beträgt typischerweise < 50 ms, was Echtzeit-Feedback während des Zeichnens ermöglicht.

8. Fazit & Ausblick

8.1 Fazit

Das Projekt zeigt, dass Echtzeit-Skizzenerkennung mit moderatem Aufwand und ohne GPU-Hardware realisierbar ist. Ein kompaktes CNN mit ~573.000 Parametern erreicht auf 100 Klassen eine Validierungsgenauigkeit von **82,3 %** — deutlich über menschlicher Baseline (~73 %) und dem Mindestziel des Projekts (80 %).

Die wichtigste Erkenntnis aus dem Vergleich beider Trainingsläufe: **Datentiefe schlägt Datenbreite**. Das Reduzieren von 150 auf 100 Klassen, kombiniert mit einer 2,5-fachen Erhöhung der Samples pro Klasse, brachte eine Steigerung von 76,5 % auf 82,3 % (+5,8 Prozentpunkte).

8.2 Ausblick

- **Mehr Daten:** 50.000 Samples/Klasse könnten weitere 2–4 % bringen (RAM-Limit beachten)
- **Tieferes Modell:** Residual Connections (ResNet-Stil) für bessere Gradientenweitergabe
- **Mehr Klassen:** Mit GPU-Training sind 200–345 Klassen realistisch
- **Stroke-basierte Modelle:** Ha & Eck (2017) zeigen, dass vektorbasierte Sketch-RNNs zusätzliche strukturelle Informationen aus der Zeichenreihenfolge nutzen können

Literaturverzeichnis

[1] Jongejan, J., Rowley, H., Kawashima, T., Kim, J., & Fox-Gieg, N. (2016). *The Quick, Draw! Dataset*. Google Creative Lab. <https://github.com/googlecreativelab/quickdraw-dataset>

[2] Ha, D., & Eck, D. (2017). *A Neural Representation of Sketch Drawings*. arXiv:1704.03477. <https://arxiv.org/abs/1704.03477>

- [3] Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. arXiv:1409.1556. <https://arxiv.org/abs/1409.1556>
- [4] Yu, Q., Yang, Y., Song, Y.-Z., Xiang, T., & Hospedales, T. (2015). *Sketch-a-Net That Beats Humans*. arXiv:1501.07873. <https://arxiv.org/abs/1501.07873>
- [5] Eitz, M., Hays, J., & Alexa, M. (2012). *How Do Humans Sketch Objects?* ACM Transactions on Graphics, 31(4), 44:1–44:10. <https://doi.org/10.1145/2185520.2185540>
- [6] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. Advances in Neural Information Processing Systems (NeurIPS), 25.